

Text-to-Cypher: An Agentic Approach

Nicolò Resta

University of Bari, Aldo Moro

February 7, 2026

Outline

- 1 Introduction & Motivation
- 2 Related Works
- 3 Proposed Approach
- 4 Experiments & Results
- 5 Conclusions

Text-to-Query: The Challenge

- **Text-to-Query:** Translating natural language questions into formal database queries.
- **Task:** Bridging the gap between informal natural language and formal languages
- **Goal:** Retrieving structured data from unstructured intent
- **Text-to-Cypher:** A specialized subset for Property Graph Databases.
- **Complexity:**
 - Graph structures are complex to express.
 - Cypher requires "visual" thinking (ASCII art style patterns).

Complexities & Core Sub-Tasks

- 1 **Relevant Concepts Identification:** Extracting key and relevant concepts such as *entities*, *relationships*, and *properties*.
- 2 **Relevant Formalisms Discovery:** Finding appropriate formal constructs (node labels, relationship types) of the underlying symbolical framework.
- 3 **Concepts-Formalisms Linking:** Mapping identified concepts to discovered formal constructs (Entity/Relationship/Property Linking).
- 4 **Formalisms Framework Exploitation:** Generating syntactically and semantically correct queries (in Cypher in our case).

Complexity

All these sub-tasks are inherently probabilistic or possibilistic in nature.

Research Landscape: Overview

Existing literature splits into two main directions:

- 1 **Benchmark Development:** Creating high-quality datasets for training and evaluation.
- 2 **Text-to-Query Solutions:** Designing architectures (Algorithmic vs. Agentic) to solve the translation task.

Our Focus

We leverage existing benchmarks to propose a novel, zero-shot agentic solution for Text-to-Cypher.

Research Landscape: Datasets

- **neo4j/text2cypher-2024v1¹** (Used):
 - **Size:** 44k+ instances (39k train, 4.8k test).
 - **Pros:** Access to 17+ live, publicly available Neo4j databases.
 - **Relevance:** Enables execution-based evaluation without local setup.
- **megagonlabs/cypherbench²** (Excluded):
 - **Size:** 10k instances (Wikidata derived).
 - **Cons:** Requires resource-intensive local deployment via Docker (64-128GB RAM).
 - **Relevance:** Too heavy for rapid experimentation.

¹[Ozsoy et al.(2024)Ozsoy, Messallem, Besga, and Minneci]

²[Feng et al.(2024)Feng, Papicchio, and Rahman]

Incomplete Solutions: The Identification-Discovery-Linking Assumption

- Most works³ assume **Identification**, **Discovery**, and **Linking** already solved providing the entire database schema without any *filtering* or *pruning*.
- This may lead to excessively large prompts that:
 - exceed the model's maximum context window.
 - pollute context with irrelevant information.
- How do systems performs Identification, Discovery, and Linking?
 - **Algorithmic** vs **Agentic** approaches.

³[D'Abramo et al.(2025)D'Abramo, Zugarini, and Torroni]

[Longwell et al.(2024)Longwell, Ali Akbar Alavi, Zarrinkalam, and Ensan]

[Feng et al.(2024)Feng, Papicchio, and Rahman]

[Ozsoy et al.(2024)Ozsoy, Messallem, Besga, and Minneci]

Algorithmic Approaches⁴

- **Pipeline:** Entity Linking → Schema Extraction → Generation.
- **Pros:** Deterministic, faster inference, easier to debug.
- **Cons:**
 - they rely on strong, domain-specific assumptions that may not generalize
 - they lack of robust disambiguation mechanisms with multiple candidates
 - Relationship and Property Linking are often ignored or oversimplified.

⁴[Yang et al.(2026)Yang, Li, Hu, Yu, and Lu]

Retrieval Architectures: Agentic Approaches⁵

- **Loop:** ReAct cycle (Thought → Action → Observation).
- **Pros:**
 - Flexible, adaptable to various domains and tasks.
 - Can handle ambiguity and multiple candidates through iterative refinement.
- **Cons:**
 - Ensuring Efficiency and Effectiveness at scale remains an open challenge.
 - Higher latency and potential for loops.

⁵[Walter and Bast(2025a), Walter and Bast(2025b)]

Similarity Techniques: The Trade-offs

Constraint: Effective retrieval of short schema terms (e.g., DataCenter, founded_at).

- **Lexical Matching** (Levenshtein, Jaro-Winkler, Prefix, etc.):
 - **Pros:** Very fast and effective for matching short terms/typos.
 - **Cons:** Semantic gaps (Synonyms, Polysemy, etc.).
- **Contextual Dense Embeddings** (BERT-based transformers):
 - **Pros:** Effective for sentence-level semantics.
 - **Cons:** Ineffective for very short de-contextualized terms + high compute cost.
- **FastText** (Subword Static Word Embeddings (S-SWEs)):
 - **Pros:** Fast lookup; captures semantics of words/subwords w/o overhead.
 - **Cons:** Large memory requirements for storing the vocabulary.
- **Model2Vec⁶** (Static Distilled Embeddings) - **Our Choice:**
 - **Pros:** Pros of FastText + memory efficiency

⁶[Tulkens and van Dongen(2024)]

System Architecture

- **Framework:** LangChain & LangGraph.
- **Core Engine:** GPT-4.1 (Zero-Shot).
- **Indexing Strategy:**
 - **Vector Store:** FAISS with Model2Vec (potion-retrieval-32M⁷). Indexes Node Labels, Relationship Types, Property Keys.
 - **Schema Index:** Inverted index for structural lookups (e.g., node-property mappings).

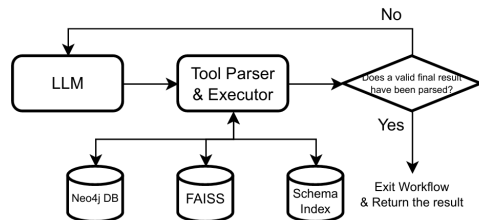


Figure: System Architecture

⁷which is a finetuned version of the potion-base-32M

Tool Design: Semantic Discovery

```
def search_for(  
    components: List[Literal["nodes", "relationships",  
        ↪ "properties"]],  
    similar_to: str,  
    top_k: int = 10  
) -> str:
```

- **Purpose:** Semantic search over indexed schema components (FAISS + Model2Vec).
- **Strategy:** Prioritizes *Recall* over Precision to cast a wide initial net.
- **Usage:** Invoked efficiently in parallel for independent concepts in the user query.

Tool Design: Property Retrieval

```
def get_properties_from(  
    components: List[Literal["nodes", "relationships"]],  
    of_types: List[str],  
    similar_to: Optional[List[str]] = None,  
    top_k: Union[int, List[int]] = 10  
) -> str:
```

- **Purpose:** Retrieves properties for explicit node labels or relationship types.
- **Batching:** Processes multiple components in a single call to reduce LLM turns.
- **Context:** Allows filtering property lists by semantic relevance (Model2Vec).

Tool Design: Structure Exploration

```
def get_domains_ranges_of(  
    relationships: List[str],  
    similar_to: Optional[List[str]] = None,  
    order_by: List[Literal["domains", "ranges"]] = None,  
    top_k: Union[int, List[int]] = 10  
) -> str:
```

- **Purpose:** Discovers valid Source (Domain) and Target (Range) nodes for relationships.
- **Graph Traversal:** essential for constructing valid path expressions.
- **Filtering:** Can prioritize certain domains/ranges based on semantic similarity to query concepts (Model2Vec).

Tool Design: Reverse Lookup

```
def get_components_with_property(  
    keys: List[str],  
    components: List[Literal["nodes", "relationships", "both"]],  
    similar_to: Optional[List[str]] = None,  
    top_k: Union[int, List[int]] = 10  
) -> str:
```

- **Purpose:** Finds schema components that possess specific property keys.
- **Disambiguation:** Uses discriminative properties (e.g., `created_at`, `isbn`) to filter candidate types.
- **Mechanism:** Leveraging the inverted schema index for efficient reverse lookups.

Tool Design: Grounding & Validation

```
def execute_cypher_query(cypher: str) -> List[dict]:
```

- **Purpose:** Executes arbitrary Cypher queries against the live database.
- **Grounding:** Validates schema assumptions against actual data instances.
- **Data Exploration:** Checks for specific patterns, value formats, or edge cases.

Tool Design: Result Registration

```
def register_retrieval_result(  
    motivation: str,  
    cypher_query: Optional[str] = None,  
    kg_schema: Optional[KGSchema] = None  
) -> RetrievalAgent.Result:
```

- **Purpose:** Signals the completion of the ReAct loop.
- **Output:** Returns specific structured data (Motivation, Query, Schema Subgraph).
- **Integration:** Passes the final result to the downstream execution system.

Experimental Setup

- **Dataset:** Subset of text2cypher-2024v1 (2,471 instances with live DBs).
- **Metrics:**
 - **Result Similarity (RS):** Jaccard similarity of result sets.
 - **Exact Match (EM):** $RS = 1.0$.
 - **Success Rate:** Completion without exceptions or loops.
 - **Provenance Subgraph Jaccard Similarity (PSJS):** Jaccard similarity of retrieved schema subgraph vs gold schema subgraph.
- **Baselines:** Comparison with Zero-shot and Fine-tuned GPT-4o (from Neo4j Labs paper).

Effectiveness Results

Comparison with best baselines of Neo4j-Labs Team⁸.
Our zero-shot agent matches fine-tuned performance.

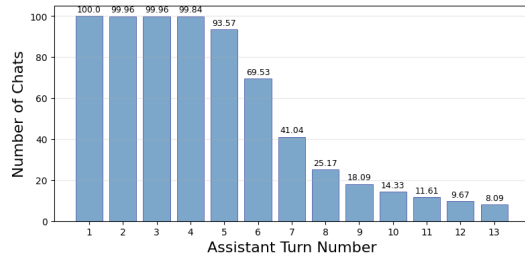
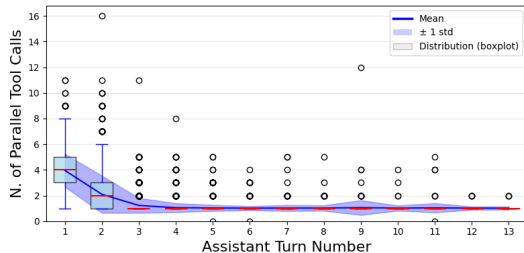
Model	Result Sim.	Exact Match	PSJS	Success Rate
GPT-4o (zero-shot)	-	31.73%	-	-
GPT-4o (fine-tuned)	-	32.50%	-	-
GPT-4.1 (Ours)	41.22%	32.62%	69.37%	89.11%

⁸[Ozsoy et al.(2024)Ozsoy, Messallem, Besga, and Minneci]

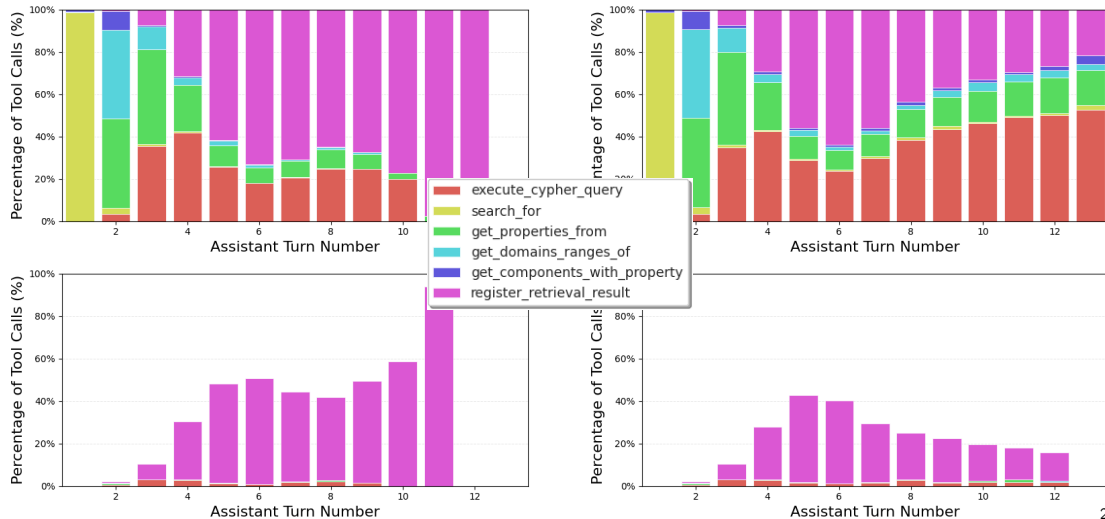
Efficiency Analysis: Absolute Tool Parallelism

Tool	Mean	Std	Mode	Min	Median	Max	Count
 execute_cypher_query	1.01	0.10	1	1	1	4	4342
 search_for	3.69	1.43	4	1	4	11	2614
 get_properties_from	1.42	0.76	1	1	1	16	3813
 get_domains_ranges_of	1.46	0.90	1	0	1	12	1924
 get_components_with_property	2.05	1.47	1	1	2	9	523
 register_retrieval_result	1.00	0.00	1	1	1	1	4659

Efficiency Analysis: Turn Parallelism and Distribution



Efficiency Analysis: Tool Calls and Errors Distributions



Conclusions

- **Agentic Approach:** Successfully bridges NL and formal queries without fine-tuning.
- **Performance:** Matches fine-tuned baselines (EM $\approx$ 32.6%).
- **Strategy:** Validated "search-then-refine" behavior with high initial parallelism.
- **Limitations:**
 - Struggle with complex, non-linear reasoning paths (backtracking).
 - JSON formatting errors in final result registration.
- **Future Work:**
 - Robust error recovery mechanisms.
 - Hybrid Fine-tuning for tool usage and formatting.

Thank You!

Questions?

n.resta6@studenti.uniba.it
<https://github.com/ashkihotah>

-  M. G. Ozsoy, L. Messallem, J. Besga, G. Minneci, Text2cypher: Bridging natural language and graph databases, 2024. URL: <https://arxiv.org/abs/2412.10064>. arXiv:2412.10064.
-  Y. Feng, S. Papicchio, S. Rahman, Cypherbench: Towards precise retrieval over full-scale modern knowledge graphs in the llm era, arXiv preprint arXiv:2412.18702 (2024).
-  J. D'Abramo, A. Zugarini, P. Torroni, Investigating large language models for text-to-SPARQL generation, in: W. Shi, W. Yu, A. Asai, M. Jiang, G. Durrett, H. Hajishirzi, L. Zettlemoyer (Eds.), Proceedings of the 4th International Workshop on Knowledge-Augmented Methods for Natural Language Processing, Association for Computational Linguistics, Albuquerque, New Mexico, USA, 2025, pp. 66–80. URL:

<https://aclanthology.org/2025.knowledgenlp-1.5/>.
[doi:10.18653/v1/2025.knowledgenlp-1.5](https://doi.org/10.18653/v1/2025.knowledgenlp-1.5).

-  J. Longwell, M. Ali Akbar Alavi, F. Zarrinkalam, F. Ensan,
Triple augmented generative language models for sparql query generation from
natural language questions,
in: Proceedings of the 2024 Annual International ACM SIGIR Conference on
Research and Development in Information Retrieval in the Asia Pacific Region,
SIGIR-AP 2024, Association for Computing Machinery, New York, NY, USA, 2024,
p. 269–273. URL: <https://doi.org/10.1145/3673791.3698426>.
[doi:10.1145/3673791.3698426](https://doi.org/10.1145/3673791.3698426).
-  C. Yang, C. Li, X. Hu, H. Yu, J. Lu,
Enhancing knowledge graph interactions: A comprehensive text-to-cypher pipeline
with large language models,

Information Processing & Management 63 (2026) 104280. URL: <https://www.sciencedirect.com/science/article/pii/S0306457325002213>. doi:<https://doi.org/10.1016/j.ipm.2025.104280>.

 S. Walter, H. Bast,
GRASP: generic reasoning and SPARQL generation across knowledge graphs,
in: ISWC (1), volume 16140 of *Lecture Notes in Computer Science*, Springer,
2025a, pp. 271–289.

 S. Walter, H. Bast,
Knowledge graph entity linking via interactive reasoning and exploration with
GRASP,
in: OM 2025 (Ontology Matching Workshop), CEUR Workshop Proceedings,
CEUR-WS.org, 2025b. To appear.

 S. Tulkens, T. van Dongen, Model2vec: Fast state-of-the-art static embeddings, 2024. URL: <https://github.com/MinishLab/model2vec>.
doi:10.5281/zenodo.17270888.